

## 제15강

### 1) Firebase Firestore Database 이용한 게시물 정보 출력, 삭제

[src]-[components15]-[post] PostRouter.jsx

```
import ReadPage from './ReadPage'
import UpdatePage from './UpdatePage'
...
const PostRouter = () => {
  return (
    <Routes>
      <Route path="/" element={<ListPage/>}/>
      <Route path="/register" element={<RegisterPage/>}/>
      <Route path="/:id" element={<ReadPage/>}/>
      <Route path="/update/:id" element={<UpdatePage/>}/>
    </Routes>
  )
}
```

[src]-[components15]-[post] ListPage.jsx

```
import { useNavigate, useSearchParams, Link } from 'react-router-dom'
...
const ListPage = () => {
  const [loading, setLoading] = useState(false);
  const navi = useNavigate();

  const db = getFirestore(app);
  const [posts, setPosts] = useState([]);
  const [search] = useSearchParams();
  const page = parseInt(search.get('page')) || 1;
  const size = 5;
  const lastRef = useRef(1);
  const onRegister = () => { ... }
  const callAPI = () => { ... }

  useEffect(()=>{
    callAPI();
  }, [page]);

  if(loading || !posts) return <h1 className='text-center my-5'>로딩중...</h1>
  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>게시글</h1>
      <div className='text-end mb-5'>
        <Button onClick={onRegister} className='px-5'>새글작성</Button>
      </div>
      <Table striped hover bordered>
        <thead> ... </thead>
        <tbody>
          {posts.map((post)=>
            <tr key={post.seq}>
              <td>{post.seq}</td>
              <td><Link to={`/${post.$post.id}`}>{post.title}</Link></td>
              <td>{post.email}</td>
              <td>{post.date}</td>
            </tr>
          )}
        </tbody>
      </Table>
      <PagingButton page={page} last={lastRef.current}/>
    </div>
  )
}
```

```

import { app } from '../initFirebase'
import { getFirestore, getDoc, doc, setDoc } from 'firebase/firestore'
...
const ReadPage = () => {
  const [loading, setLoading] = useState(false);
  const { setConfirm } = useContext(ModalContext);
  const navi = useNavigate();

  const db = getFirestore(app);
  const { id } = useParams();
  const [post, setPost] = useState('');
  const { title, body, email, date } = post;

  const callAPI = async() => {
    setLoading(true);
    const snapshot = await getDoc(doc(db, 'posts', id));
    setPost({ id:snapshot.id, ...snapshot.data() });
    //console.log(snapshot.id, snapshot.data());
    setLoading(false);
  }
  useEffect(()=>{
    callAPI();
  }, []);

  const onClickDelete = () => {
    setConfirm({
      show:true,
      message:`${id}번 문서를 삭제하실래요?`,
      action: async()=>{
        await deleteDoc(doc(db, 'posts', id));
        navi('/post');
      }
    });
  }

  const onClickUpdate = () => {
    navi(`/post/update/${id}`);
  }

  if(loading) return <h1 className='text-center my-5'>로딩중...</h1>
  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>게시글 정보</h1>
      <Row className='justify-content-center'>
        <Col md={10} lg={9} xl={8}>
          { sessionStorage.getItem('uid') && <div className='text-end mb-2'>
            <Button onClick={onClickUpdate} className='px-3 me-2' variant='outline-primary'>수정</Button>
            <Button onClick={onClickDelete} className='px-3' variant='outline-danger'>삭제</Button>
          </div> }
        <Card>
          <Card.Header>
            <h5 className='my-2'>{title}</h5>
          </Card.Header>
          <Card.Body>
            <div style={{whiteSpace:'pre-wrap'}}>{body}</div>
          </Card.Body>
          <Card.Footer className='text-muted'>
            Posted on <span>{date}</span> by <span>{email}</span>
          </Card.Footer>
        </Card>
      </Col>
    </Row>
  </div>
)
}

```

## 2) Firebase Firestore Database 이용한 게시물 수정

[src]-[components15]-[post] Update.jsx

```
import { app } from '../initFirebase'
import { getFirestore, getDoc, doc, setDoc } from 'firebase/firestore';
...
const UpdatePage = () => {
  const [loading, setLoading] = useState(false);
  const { setConfirm } = useContext(ModalContext);
  const navi = useNavigate();

  const { id } = useParams();
  const db = getFirestore(app);
  const [form, setForm] = useState('');
  const { title, body } = form;
  const onChange = (e) => {
    setForm({...form, [e.target.name]:e.target.value});
  }

  const formRef = useRef(null);
  const callAPI = async() => {
    setLoading(true);
    const snapshot = await getDoc(doc(db, 'posts', id));
    const post = { id:snapshot.id, ...snapshot.data() };
    //console.log(post);
    setForm(post);
    formRef.current=post;
    setLoading(false);
  }
  useEffect(()=>{
    callAPI();
  }, []);

  const onSubmit = (e) => {
    e.preventDefault();
    setConfirm({
      show:true,
      message:`${id}번 게시글을 수정하실래요?`,
      action: async()=>{
        await setDoc(doc(db, 'posts', id), form);
        navi(-1);
      }
    });
  }

  if(loading) return <h1 className='text-center my-5'>로딩중...</h1>
  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>게시글 수정</h1>
      <Row className='justify-content-center'>
        <Col md={10} lg={9} xl={8}>
          <Form onSubmit={onSubmit}>
            <Form.Control value={title} onChange={onChange}
              name='title' placeholder='제목을 입력하세요.' className='mb-3' />
            <Form.Control value={body} onChange={onChange}
              name='body' placeholder='내용을 입력하세요.' className='mb-3' rows={10} as='textarea' />
            <div className='text-center'>
              <Button disabled={JSON.stringify(form)===JSON.stringify(formRef.current)}
                type='submit' className='px-5 me-2'>저장</Button>
              <Button disabled={JSON.stringify(form)===JSON.stringify(formRef.current)}
                type='reset' className='px-5' variant='secondary'>취소</Button>
            </div>
          </Form>
        </Col>
      </Row>
    </div>
  )
}
```